

# A Comprehensive Guide to LLM Agent Debugging

Gemini Agent

An Exhaustive Walkthrough of AgentDebug (Zhu et al., 2025)

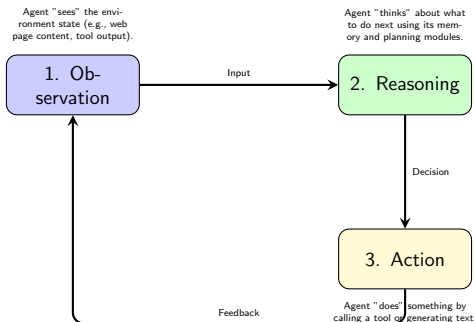
November 13, 2025

# Presentation Agenda

- 1 Understanding the Agent's World: Trajectories & Failures
- 2 A Deep Dive into the AgentErrorTaxonomy
  - ▶ Methodology
  - ▶ Complete breakdown of all 5 modules and their error types
- 3 The AgentDebug Framework: A Step-by-Step Guide
- 4 Experimental Validation: Does it Work?
- 5 Illustrative Cases from Multi-Agent Systems (MAST)
- 6 Conclusion & Key Takeaways

# The Agent's Reasoning Loop

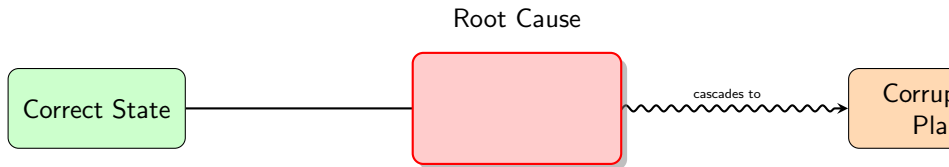
An LLM Agent operates in a continuous cycle. A failure in any part of this loop can derail the entire task.



**Trajectory:** The full, step-by-step log of this cycle for a given task. When we debug, we are analyzing this trajectory.

# The Core Problem: Cascading Error Propagation

The primary bottleneck in LLM agent reliability is **error propagation**. A single, minor error early in a task can corrupt all subsequent steps.



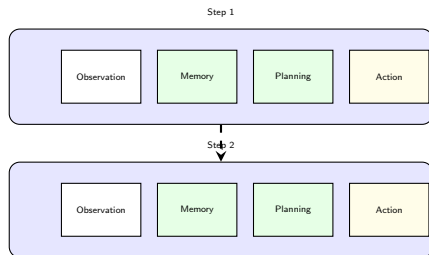
## Example: The Wrong Portland

- ❶ **Root Error (Memory):** Agent misreads an email, recalling destination as "Portland, **Maine**".
- ❷ **Cascade (Planning & Action):** It correctly books a flight and hotel... but in the wrong city.
- ❸ **Result (Task Failure):** The agent reports success, but the user is left with a useless itinerary.

To build robust agents, we must move beyond just observing failure; we need a principled way to find and fix the **root cause**.

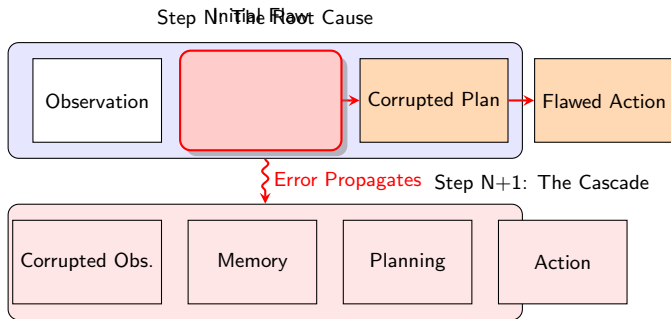
# Visualizing an Agent's Trajectory

A **trajectory** is the complete log of an agent's attempt to solve a task. It's a sequence of steps, where each step contains the agent's entire reasoning process.



# How and Where Failures Appear in a Trajectory

A single flaw in one module can corrupt the entire rest of the step, and cascade into future steps.



# Building a Language for Failure

Before we can debug, we need a precise vocabulary to describe what went wrong.

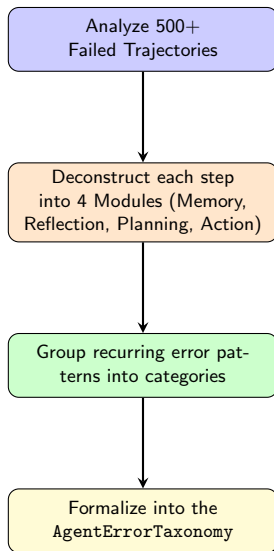
## **Methodology (from Zhu et al., 2025):**

- Analyzed over 500 failed agent trajectories from diverse benchmarks (ALFWorld, WebShop, GAIA).
- Decomposed each agent's turn into four operational modules: **Memory, Reflection, Planning, Action.**
- Grouped recurring failure patterns into a structured taxonomy.

The result is the `AgentErrorTaxonomy`, a comprehensive classification of how and where agents fail.

# Building a Language for Failure: Methodology

To debug failures, we first need a precise vocabulary to describe them.





# The Full AgentErrorTaxonomy (1/3): Memory

**Memory:** Errors in recalling or retrieving information.

**Over-simplification / Incomplete Summary** Summarizes past info too crudely, ignoring critical details.

- *Example:* Agent is told "find a red shirt, size medium." Its memory becomes "find a shirt," forgetting the constraints.

**Hallucination (False Memory)** Recalls events or states that never happened.

- *Example:* After a failed API call, the agent "remembers" that the call was successful and returned data.

**Retrieval Failure** Relevant information exists in the agent's history, but it fails to retrieve it when needed.

- *Example:* The user provided an API key on step 1, but on step 5 the agent asks for the key again.

# The Full AgentErrorTaxonomy (2/3): Reflection & Planning

**Reflection:** Errors in monitoring progress or interpreting outcomes.

**Progress Misassessment** Incorrectly evaluates its own progress (e.g., thinks it's done when it's not).

**Outcome Misinterpretation** Correctly executes an action but misreads the feedback from the environment.

**Causal Misattribution** Correctly notes a failure but blames the wrong cause.

**Planning:** Errors in formulating a strategy.

**Constraint Ignorance** Ignores limits (time, budget, user rules) when forming a plan.

**Impossible Action** Plans a step that is physically or logically impossible.

**Inefficient Planning** The plan is overly long, illogical, or wastes steps.

# The Full AgentErrorTaxonomy (3/3): Action & System

**Action:** Errors in executing an operation.

**Planning-Action Disconnect** The chosen action does not align with the agent's stated plan.

**Format Error** Produces a syntactically invalid action (e.g., malformed JSON).

**Parameter Error** Generates an action with unreasonable or malformed parameters.

**System-level:** Failures caused by the external environment or tools.

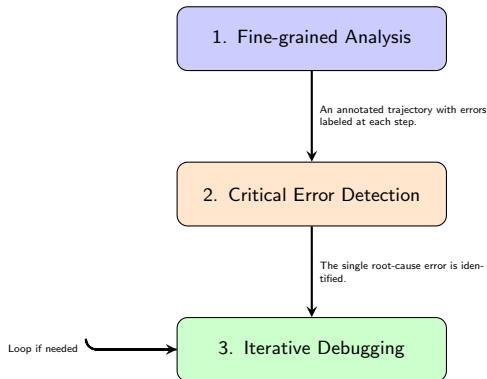
**Step Limit Exhaustion** Fails by reaching the maximum step count.

**Tool Execution Error** An external tool/API crashes or returns an error.

**Environment Error** The environment itself breaks or does not follow expected rules.

# The AgentDebug Framework: A Step-by-Step Guide

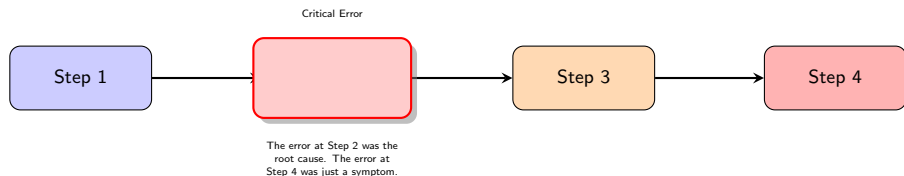
With a taxonomy in place, we can build a system to automatically find and fix errors.



# Framework Deep Dive: Critical Error Detection

This is the core of AgentDebug. It's not about finding all errors, but about finding the **one** that mattered most.

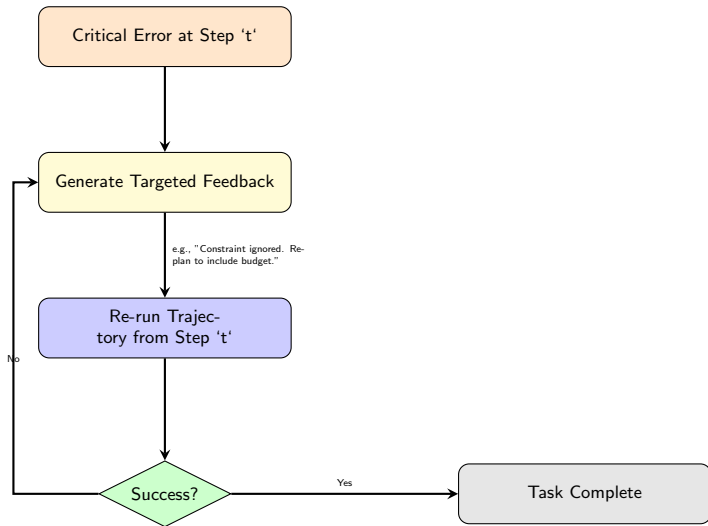
**The "Counterfactual" Question:** The system asks for each step 't' in the trajectory: *"If the agent had done the right thing at step 't', would the rest of the trajectory have succeeded?"*



The **Critical Error** is the *earliest* step where the answer is "Yes." This prevents wasting time fixing downstream symptoms of an earlier, deeper mistake.

# Framework Deep Dive: Iterative Debugging

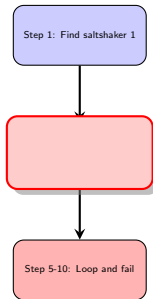
Once the root cause is found, AgentDebug generates targeted feedback and re-runs the agent from that critical point.



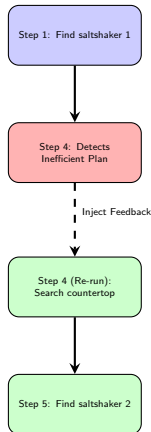
# AgentDebug in Action: A Visual Example

**Task:** Find two saltshakers and put them in a cabinet.

## Normal Agent (Fails)



## With AgentDebug (Succeeds)



**Feedback Injected:** "The agent fixated on cabinets, ignored likely surfaces; fix by expanding early to countertops/tables."

# Experimental Validation: Does It Work?

Yes. The paper shows that this principled debugging approach significantly improves task success compared to baselines.

**The Takeaway:** Precisely identifying and correcting the root-cause failure is far more effective than unguided self-reflection or simply trying more times.



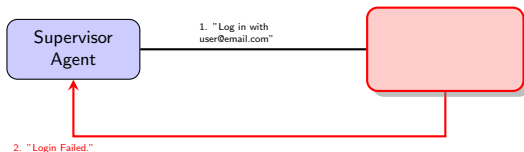
# A Glimpse into Multi-Agent Failures

While AgentDebug focuses on single agents, the MAST paper (Cemri et al., 2025) provides great examples of failures unique to **multi-agent systems**.

In MAS, failures often stem from flawed **communication** and **coordination**, not just individual reasoning.

# MAS Failure Case: Information Withholding

**Category:** Inter-Agent Misalignment (Agents fail to coordinate effectively).

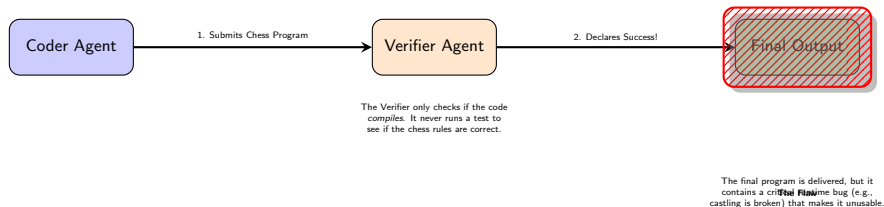


The Login Agent knows the API requires a *phone number*, not an email, ~~The~~ ~~file~~ username. It withholds this critical information, causing repeated failures.

This is a communication failure. The system as a whole fails because the agents do not share necessary context to help each other succeed.

# MAS Failure Case: Incorrect Verification

**Category:** Task Verification (The system fails to properly check its own work).



The system prematurely declares success because its verification process was too superficial.

# Conclusion & Key Takeaways

## ❶ **Failure is Systemic, Not Just Local**

Errors are rarely isolated. They propagate within a single agent's reasoning and emerge from flawed interactions between multiple agents.

## ❷ **A Taxonomy is a Prerequisite for Debugging**

We need a precise language to describe failures before we can build tools to fix them. The `AgentErrorTaxonomy` provides this for single agents.

## ❸ **Root Cause Analysis is Essential**

Fixing surface-level symptoms is not enough. Effective debugging, as demonstrated in `AgentDebug`, must find and correct the single, earliest point of failure.

## ❹ **Automation is the Goal**

Frameworks like `AgentDebug` are the first step towards creating agents that can learn from their own mistakes and become truly robust and reliable.

# References I



Zhu, K., Liu, Z., Li, B., et al. (2025).

*WHERE LLM AGENTS FAIL AND HOW THEY CAN LEARN FROM FAILURES.*

[arXiv:2509.25370v1](#).



Cemri, M., Pan, M. Z., Yang, S., et al. (2025).

*Why Do Multi-Agent LLM Systems Fail?*

[arXiv:2503.13657v3](#).

Thank You

**Questions?**